EXASOL HACKATHON · TRACK 2 · CUSTOMERS, CHANNELS & DEMAND

Ripple

The full bill of every stock-out.

Catch phantom stock-outs in a day and a half, then follow every affected shopper through the return, the ticket and the quiet exit, all inside Exasol Personal.



Debi Prasad Samal · Naveen Kumar · Nekita Saras Dhanabal

github.com/debi910/Ripple

THE PROBLEM

Quick-commerce dark stores promise ten minutes from stock records that are often wrong.

$1.2T

a year lost to out-of-stocks, inside $1.7T of inventory distortion

IHL Group

2 in 3

store stock records are wrong, in a US chain (2008) and a UK grocer (2025). Phantom stock never raises an alert.

DeHoratius & Raman; arXiv 2506.05357

82.5%

Zepto’s in-stock rate this month, losing 11 points through the day (Blinkit: 94.4%)

Bernstein, Oct 2026

Who feels it: grocers and quick-commerce players, whose shoppers get a swapped item or a refund instead of what they paid for, and quietly switch apps.

THE INSIGHT

Track 2 lists four problems. In real life they are one chain.

1

Shelf empties

The system still says in stock. Nobody is told.

2

Order breaks

Paid item is swapped or refunded at picking.

3

Return

The substitute comes back as "not as expected".

4

Ticket

Written from a second email or a WhatsApp number.

5

Exit

The shopper goes quiet. Nobody links it to step 1.

Stock-outs found too late, returns that erode margin, churn hiding in tickets, one customer seen differently by each system: Ripple follows the whole chain.

THE GAP

Every tool sees one system

Tool category	Sees	Misses
Demand planning	Stock records, forecasts	"App shows in stock but never delivers"
Returns platforms	Return flow, coarse reason codes	That the return was a substitute for a phantom
Helpdesk AI	Ticket sentiment, routing	The same person's missing item under another email
CDPs	Merged profiles for marketing	Links from people back to stock events

So a stock-out is costed as **one lost sale**. The rest of the bill is split across three other ledgers and never added up.

HOW IT WORKS

Five source systems, four engines, one ledger

- Inventory book + counts
- POS daily sales
- Orders, picks, accounts
- Support tickets
- Returns



Phantom filter
Hidden Markov filter per SKU-store, Python SET UDF

Identity graph
Fuzzy match + basket fingerprint, Lua closure

Ticket reader
Intent and exact SKU, Python UDFs

Causal churn
Matched natural experiment in SQL



Ripple ledger
Every rupee a receipt. Recount list. Proof.

All data and all heavy logic stay inside Exasol Personal. Python outside only generates data, sends SQL and serves the page.

ENGINE 1 · EARLY WARNING

Joining the silos is what catches the phantom

Same filter, fed fewer systems. Share of 504 injected phantoms caught, and days from empty shelf to alert:



14.2

days before the stock record would have been corrected

86%

of alerts were real phantoms

Only 4% of affected shoppers complain, as published research says, so tickets add little: short picks and picker logs carry it. Calibrated from past cycle counts, never the answer key.

ENGINES 2 AND 3 · WHO IT HIT

One shopper, many accounts, one angry ticket

Identity graph

99.9%

of merged account pairs are truly one person (83% recall). 70,107 accounts become 44,541 shoppers.

Gmail dots, phone formats, swapped and typo'd names, a household-phone guard and a purchase-basket fingerprint. A Lua script closes the graph inside Exasol.

Ticket reader

96.3%

intent accuracy across eight intents, Hinglish and typos included. 85% of complaints resolved to the exact SKU.

“toned milk nahi aaya order me” only becomes a SKU once we know who wrote it and what they just ordered.

ENGINE 4 · CAUSAL CHURN

Phantoms strike at random, so they are a natural experiment

Two shoppers at the same store, with the same habits, order in the same week. One hits a silent stock-out; one gets everything.

Comparing them over the next eight weeks isolates the churn the stock-out **caused**, not the churn that happens to follow it.

Stacked difference-in-differences, matched on store, prior order frequency and basket size, all in SQL.

A missing item makes a shopper

+4.6 pts

likelier to be gone eight weeks later (±1.5)

The true effect in the simulation is +12.2 pts. Our estimate is cautious on purpose, so the bill never overclaims.

THE LEDGER

542 phantom stock-outs, the whole bill

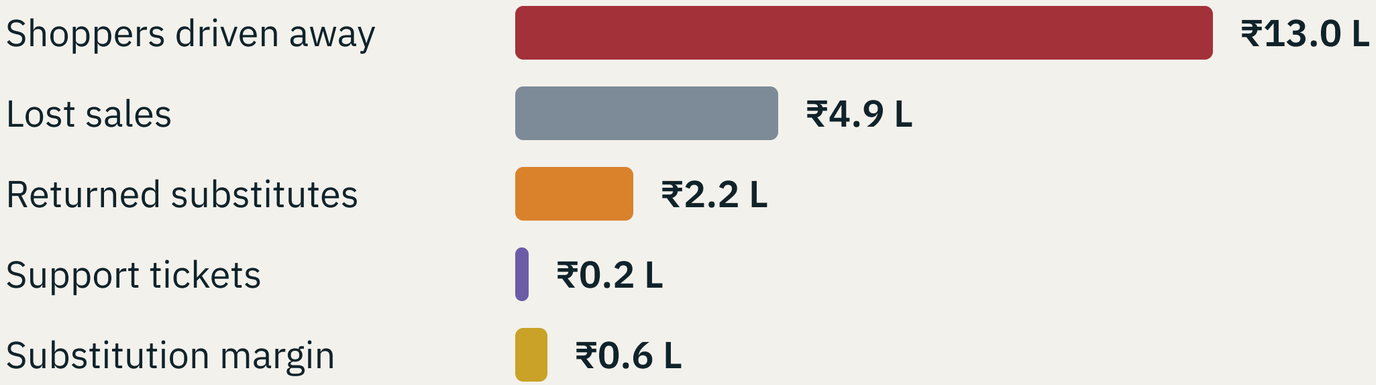
A lost-sales report shows

₹4.9 L

Ripple's ledger shows

₹21.0 L

4.3× the cost anyone sees



16,421 shopper-hits · 2,427 substitutes returned · 535 tickets. Every line is a receipt pointing at its order line, return or ticket.

THE ACTION

Every morning: which shelf to count first, ranked by rupees saved

#	Count	Store	Next count	Saving
1	Neelgiri Ultra Thin Pads 8 pcs	Viman Nagar	30 days	₹26,836
2	Pinewood Green Peas 1kg	HSR Layout	40 days	₹23,680
3	Neelgiri Baby Diapers M 56 pcs	Kondapur	14 days	₹9,508
4	Lotus Leaf Instant Coffee 100g	Kothrud	27 days	₹8,412

Expected saving = P(phantom) × daily demand × cost per broken line × days until the scheduled count. Each alert shows which signals drove it and the customers' own words. The app replays any past morning using only data available then.

THE MAKE-GOOD

Reach the shoppers who never complain, before they leave

Only about 1 in 26 unhappy customers complains. Ripple ranks the silent shoppers a phantom hit by the money at stake and drafts one offer each:

> “Sorry your Gokul Fresh Basmati Rice 5kg didn’t arrive on 05 Apr. We’ve added Rs 90 to your account. If it’s ever short again, we’ll send Chandra Basmati Rice 5kg instead.”

Designed from field evidence: a small credit, never a bare apology (Uber, 1.5M riders); one offer per shopper; repeat hits first (Knox & van Oest); no blanket discounts (Anderson et al.); a substitute they already trust (Hoang & Breugelmans).

2,004
silent shoppers worth an offer, of 11,945

₹1.3 L → ₹1.9 L
credit offered → margin expected back

32% vs 23%
of listed vs other silent shoppers really left

WHY EXASOL

The hard parts run inside the database

Python SET UDF with GROUP BY + ORDER BY: one HMM filter per SKU-store, three ablation variants per pass

Python scalar UDFs read ticket intent and match products

Lua script runs label propagation to a fixed point with MERGE

SQL: EDIT_DISTANCE, SOUNDEX, TF-IDF, window functions, difference-in-differences

IMPORT bulk load; every app view is a live query

Large world on one 8 GB laptop

30 stores · 200k shoppers · 180 days

Step	Time
Load 12M rows	32 s
Identity, 350k accounts	15 s
Tickets, 49k	11 s
Detector, 1.9M SKU-days ×3	4¼ min
Causal, ledger, make-good, scoring	22 s

At scale: 92% of phantoms caught in 1.3 days; identity merges 99.8% correct.

AGAINST THE LITERATURE

Ripple beats the textbook methods on the same answer key

Detection

	Caught	Days	AUC
Ripple	91%	1.6	0.99
Zero-sales rule*	49%	15.0	0.97

*Kang & Gershwin’s N-zero-days rule, best N (17) picked in hindsight. Farias et al. report AUC 0.67 on real data.

Which shelf to count (3 per store per day)



Stress test: with only 2% to 25% of affected shoppers complaining, detection stays between 87% and 91%.

PROOF AND OPENNESS

Every claim is scored against an answer key Ripple never reads

Seeded world

A day-by-day simulator writes the five source systems plus the truth: real phantom intervals, real owners, real churn. Same seed, same numbers.

Live Proof tab

Precision, recall, lead time, ablation, the causal check and stage timings, queried from Exasol in the app.

One command

python -m ripple.build generates, loads and runs every engine. MIT licence, synthetic data only, invented names.

Ledger check: 98.7% of the order lines real phantoms broke are on the bill, and 98.6% of billed lines were broken by a real phantom.

A stock-out is not one lost sale.

Ripple shows the whole bill, says which shelf to count tomorrow, and proves both.

github.com/debi910/Ripple · built on Exasol Personal

Debi Prasad Samal · Naveen Kumar · Nekita Saras Dhanabal

